Initial Details

For H&T, I worked as a Backend .NET Developer. H&T is a pawnbroking company that operates multiple retail stores where customers pledge valuables such as gold or jewelry in exchange for loans. The core in-store system used by the business is called **EVO**, which is built on a microservices-based architecture. Through EVO, store staff can create loans against pledged items, process repayments, handle partial paydowns, renew loans, and manage various customer-related operations.

Problem

However, one major limitation was that customers had to physically visit the store for almost every activity, whether it was checking loan details, making a payment, or renewing a loan.

Solution

To solve this problem and improve customer convenience, we worked on a new application called **Digital Customer Portal.** This was designed as a Single Page Application developed in React, accessible via both web and mobile browsers. Since EVO was already built using **microservices architecture**, we followed the same pattern for the new portal to maintain consistency and scalability. For authentication and authorization, we integrated Azure AD B2C for secure login and token-based authentication.

From a backend design perspective, we followed **Repository Pattern** and **Unit of Work.** We implemented the **CQRS MediatR Pattern**, manage multi-step transactions across services and use the **choreography Saga design pattern**. We also applied the **Circuit Breaker pattern (POLLY)** to prevent cascading failures in case dependent services became unavailable.

For API exposure, we used **Azure API Management** as an API gateway to centralize API access, apply security policies, enable rate limiting. We used **Cloudflare** for CDN, SSL termination, and Web Application Firewall capabilities to enhance performance and security at the edge. Microservices communicated asynchronously using **Azure Service Bus**. Sensitive configuration data such as connection strings and keys were stored securely in **Azure Key Vault.** We also used **Azure Feature Flags** to control the rollout of new features, especially since the Digital Customer Portal required modifications in some of the existing services.

Azure Storage for storing documents and receipts, **Azure Functions** for background processing and synchronization tasks, and **Azure Application Insights** for logging, monitoring, and telemetry. For the database layer, we used **SQL Server** for core transactional financial data and **Cosmos DB** for scalable, document-based storage where high availability and flexible schema were required.

For payment processing, we integrated with Checkout.com as the payment gateway. We integrated SendGrid to send emails. From a quality perspective, we wrote unit test cases using **xUnit.** For CI/CD and release management, we used Azure DevOps pipelines to automate builds, deployments, and environment-based configurations.